

Week 03 - Lab 2

Arena Survival: STL Enemy Wave Challenge

Duration	2 hours
Lab Type	Independent / ability-testing practical
Context	Game development - arena survival and enemy wave management
Submission	One .cpp file
Difficulty	Moderate to challenging

Challenge Brief

You are developing the enemy-management system for an arena survival game. The player faces several waves of enemies, takes tactical decisions, searches for targets, heals critical units, removes defeated enemies and monitors the vector as it grows. Unlike Lab 1, this lab provides only minimal starter code. You must decide which vector/STL operation is appropriate for each requirement.

Important rules

Use the STL features requested in each task. When a task says use `find_if()`, do not replace it with a manual search loop. When removing defeated enemies, use `remove_if()` + `erase()` rather than repeated `erase()` calls. Do not assume a fixed vector capacity growth pattern.

Learning Outcomes

- Use `std::vector` to store and manage game entities.
- Apply `push_back()`, `insert()`, `erase()`, `size()`, `capacity()`, `reserve()` and `resize()` in a game scenario.
- Traverse and modify vector elements using iterators.
- Use `find_if()` with lambdas and captured values to search by condition.
- Recognise when vector operations may invalidate iterators.
- Remove defeated game entities using `remove_if()` and the erase-remove idiom.
- Combine several STL concepts to solve a small game-management problem independently.

Starter Code

Copy the following starter code. All remaining logic must be completed by you.

```
#include <iostream>
#include <vector>
#include <string>
#include <algorithm>
#include <iomanip>

using namespace std;

struct Enemy
{
    int id;
```

```

    string name;
    int health;
    int score;
};

void displayEnemies(const vector<Enemy>& enemies)
{
    cout << left
         << setw(8) << "ID"
         << setw(16) << "Name"
         << setw(10) << "Health"
         << setw(10) << "Score" << "\n";

    cout << string(44, '-') << "\n";

    for (const Enemy& e : enemies)
    {
        cout << left
             << setw(8) << e.id
             << setw(16) << e.name
             << setw(10) << e.health
             << setw(10) << e.score << "\n";
    }
}

```

What is provided?

The Enemy structure and displayEnemies() are supplied only to keep the focus on vector operations and STL algorithms. You must create the vector, add data, search, modify, remove and manage capacity yourself.

Part 1 - Create the Arena and Spawn Wave 1

Inside main(), create an empty vector of Enemy objects named enemies. Before adding the first wave, reserve space for 10 enemies.

```

vector<Enemy> enemies;
// Reserve enough capacity for the expected first part of the battle.

```

Add the following enemies using push_back():

ID	Enemy	Health	Score
201	Goblin	45	120
202	Orc	80	300
203	Skeleton	18	220
204	Dark Mage	35	500
205	Troll	100	450
206	Assassin	12	700
207	Slime	5	80
208	Dragon	150	1200

After spawning the wave:

- Call displayEnemies(enemies).
- Print the current size and capacity.
- Calculate and print the unused capacity using capacity() - size().

- Print whether one more enemy can be added without requiring additional capacity.

Ability check

Do not hard-code the answer to the last question. Your program must determine it by comparing `size()` and `capacity()`.

Part 2 - Reinforcement Wave and Capacity Behaviour

A reinforcement wave arrives. Add the following enemies one at a time using `push_back()`:

ID	Enemy	Health	Score
209	Warlock	70	650
210	Wolf	35	180
211	Knight	90	600
212	Necromancer	55	900

For each reinforcement, record the capacity before `push_back()` and compare it with the capacity afterwards.

- Display the enemy name being added.
- Display size before and after the insertion.
- Display capacity before and after the insertion.
- If capacity changes, print a message such as: Reallocation occurred.

Important

You initially reserved 10 elements, but this wave takes the vector beyond that amount. Observe what your implementation actually reports. Do not assume capacity will double.

Part 3 - Player Target Search by ID

Ask the player to enter an enemy ID. Use `std::find_if()` to locate the matching enemy. A manual search loop is not allowed.

```
int targetID;
cout << "Enter enemy ID to target: ";
cin >> targetID;

// Use find_if() here.
// Your lambda must be able to use targetID.
```

If the enemy exists, display its ID, name, health and score. Otherwise display: Target not found.

Concept being tested

The lambda needs a value that was created outside the lambda. Decide what should appear inside the capture list [...].

Part 4 - Iterator Mission: Arena Hazard

A temporary arena hazard damages every living enemy by 5 health points. For this task, use an explicit iterator loop - not an index loop and not a range-based for loop.

```
for (auto it = enemies.begin(); it != enemies.end(); ++it)
{
    // If the enemy is alive, reduce its health by 5.
}
```

- Do not allow the hazard itself to remove enemies.
- Health may become 0 or negative; cleanup happens later.
- After the hazard, call `displayEnemies(enemies)`.

Part 5 - Critical Enemy Detection

Use `find_if()` to locate the first living enemy in critical condition.

- The enemy must still be alive: `health > 0`.
- The enemy must have health below 20.
- Both conditions must be true at the same time.

If found, display the enemy name, health and score. If no such enemy exists, display: No critical enemy found.

Remember

`find_if()` returns the first element that satisfies the predicate. It does not return every matching enemy.

Part 6 - High-Value Vulnerable Target

The player wants to prioritise a valuable but vulnerable enemy. Use `find_if()` to locate the first enemy satisfying all of these conditions:

- `health > 0`
- `health < 30`
- `score >= 500`

If found, print: High-value vulnerable target detected! followed by the enemy details.

Logic test

This task tests whether you can combine more than one condition inside a lambda. Think carefully about `&&` and the order in which you express the rules.

Part 7 - Emergency Healing Decision

Find the first living enemy whose health is below 25 and whose score is at least 500. If such an enemy exists:

1. Store its old health.
2. Increase its health by 20.
3. Do not allow the new health to exceed 100.
4. Display the enemy name, old health and new health.

If no enemy matches, display: No emergency healing required.

Part 8 - Boss Arrival and Iterator Safety

Before adding the boss, create an iterator that points to the first enemy:

```
auto selectedEnemy = enemies.begin();
```

Now create the boss:

```
Enemy boss {999, "Ancient Dragon", 250, 5000};
```

- Insert the boss at `enemies.begin()`.
- Display the vector after insertion.
- Do NOT dereference `selectedEnemy` after the insertion.
- Write a comment explaining why it would be unsafe to assume that `selectedEnemy` is still valid.
- Remove the boss using `erase(enemies.begin())` and display the vector again.

Safety rule

`insert()`, `erase()`, and `push_back()` operations that cause reallocation may invalidate iterators. This task is testing your understanding, not asking you to crash the program.

Part 9 - Defeated Enemy Cleanup

After the arena hazard and player actions, some enemies may have health ≤ 0 . Remove all defeated enemies using the erase-remove idiom.

- Store the vector size before cleanup.
- Use `remove_if()` to identify defeated enemies.
- Use `erase()` to physically remove the unwanted tail.
- Calculate how many enemies were removed.
- Display the remaining enemies.

```
// Your solution should follow this overall idea:
// 1. call remove_if(...)
// 2. use the returned logical end
// 3. erase the unwanted range
```

Part 10 - Spawn Slot Experiment: `reserve()` vs `resize()`

The game engine now experiments with a separate vector that stores integer spawn-slot values.

```
vector<int> spawnSlots;
```

5. Call `spawnSlots.reserve(5)`. Print size and capacity.
6. Without adding any values, answer in a comment: Does `spawnSlots[0]` exist yet?
7. Call `spawnSlots.resize(5)`. Print size, capacity and all elements.
8. Answer in a comment: Why is `spawnSlots[0]` now valid?
9. Explain in one sentence the difference between `reserve()` and `resize()`.

No guessing

Print the actual capacity reported by your program. The important concept is that `reserve()` prepares storage, while `resize()` changes the number of actual elements.

Part 11 - Custom Tactical Search

Ask the player to enter two values:

- minimum score
- maximum health

Use `find_if()` to locate the first living enemy satisfying all of the following:

- `score >=` minimum score entered by the player
- `health <=` maximum health entered by the player
- `health > 0`

Capture test

Your lambda needs access to two variables created outside the lambda. Decide how both values should be captured.

Part 12 - Final Challenge: Prepare the Next Wave

The game is about to spawn 6 more enemies. Before adding them, your program must decide whether the current capacity is sufficient.

- Calculate how many unused slots are currently available.
- If unused capacity is less than 6, reserve enough space for the current size + 6 elements before spawning the wave.
- Display size and capacity before the reserve decision.
- Display size and capacity after the reserve decision.
- Explain in a short comment why `reserve()` is more appropriate here than `resize()`.

Then add the following final wave:

ID	Enemy	Health	Score
301	Wraith	30	520
302	Golem	120	800
303	Imp	10	90
304	Vampire	60	950
305	Hunter	75	650
306	Demon	95	1300

Finally, print a battle summary containing:

- number of active enemies
- current vector size
- current vector capacity
- unused capacity
- name of the first living enemy with health below 20, if one exists
- name of the first enemy with score above 1000, if one exists

Restrictions

- Do not use a manual index search where `find_if()` is explicitly required.
- Do not remove defeated enemies one-by-one; use `remove_if()` + `erase()`.
- Do not deliberately dereference an iterator after a vector-changing operation if it may be invalid.
- Do not assume a fixed capacity growth rule.
- Your program must compile and run without errors.

Final Checklist

- Wave 1 and reinforcement enemies are stored in `vector<Enemy>`.
- `reserve()`, `size()` and `capacity()` are used correctly.
- At least one task uses an explicit iterator to modify enemies.
- `find_if()` is used with single-condition and multi-condition lambdas.
- A captured lambda is used for player input.
- Iterator invalidation is explained correctly.
- `remove_if()` + `erase()` performs defeated-enemy cleanup.
- `reserve()` and `resize()` are distinguished in the spawn-slot experiment.
- The final wave capacity decision is computed rather than hard-coded.
- The final program is readable, indented and uses short comments where requested.

End of Lab

This lab is intended to test whether you can choose and combine the Week 03 vector/STL techniques independently. The objective is not only to make the program work, but to use the correct STL operation for each game-management problem.